AI Identity Security

Capstone Project

Project 1: Threat Modelling the AI Identity System

Submitted by

Sri Vidya Praveena

Project Description

SecureNova Inc. runs an AI-powered customer service platform. Instead of a human handling every support request, an AI agent (built on an LLM) reads the customer's question and takes action on its own — it looks up information from a knowledge base (the RAG pipeline), calls internal company systems (REST APIs), and uses tools through an MCP server to do things like check an order or update a ticket.

The problem is that this AI agent isn't just a chatbot — it behaves like a user with its own login and its own permissions. It authenticates through Auth0, it holds an API key to talk to the LLM provider, and it uses an OAuth client to call internal APIs on the customer's behalf. The AI agent, the API key, the OAuth client, the RAG pipeline, and the MCP server are each their own identity in this system, and each one can fail in a different way: a credential can be stolen or leaked, an identity can be spoofed, or an identity can simply be given more access than it needs. Any one of these failures could let an attacker steal data, manipulate the AI's responses, or move deeper into SecureNova's internal systems.

Before any of these risks can be fixed, they first need to be found and understood. That is what Project 1 does: it identifies every identity in the platform, maps out exactly how data and requests flow between them, and systematically works through what could go wrong at each step. The output of this project — the identity inventory, the data-flow diagram, the STRIDE matrix, the attack trees, and the risk register — is the foundation the rest of the capstone is built on.

Objectives

This project has four objectives:

- List every identity in the system human and non-human along with how each one authenticates and what it is allowed to do.
- Build a data-flow diagram in OWASP Threat Dragon showing how a request travels through the system, with clear trust boundaries between the user, the agent, and the internal APIs.
- Apply STRIDE (a standard threat-categorization method) to every flow in that diagram, to surface specific ways each identity or connection could be attacked, and build three attack trees for the platform's most concerning attack paths.
- Map each attack tree to a real, documented AI attack technique from MITRE ATLAS, and produce a risk register ranking every identified threat by likelihood and impact.

Data-Flow Diagram with Trust Boundaries (Screenshot-1)

I built the data-flow diagram below in OWASP Threat Dragon. It shows how a request travels from the customer all the way down to the database, split across three trust boundaries: the User layer, the Agent layer, and the Internal API layer. Every line crossing a dashed boundary is a trust boundary crossing -a point where data or a request moves from one zone of trust into another, and therefore a point that needs to be defended and threat-modelled.

What I built

- User layer: the Human user sends an HTTP request to the Web app, which forwards an authentication request to Auth0. Auth0 is drawn as an Actor (external identity provider, not owned by Secure Nova).
- Auth0 issues a JWT back to the AI agent, and the Web app separately forwards the customer's query plus that JWT into the Agent layer - two distinct arrows, because the token and the request travel through different paths before meeting at the AI agent.
- Agent layer: the AI agent fetches the LLM API key from a dedicated key store, sends scoped queries to the RAG pipeline (which retrieves chunks from a Vector DB), and exchanges a client credential with the OAuth M2M client to obtain a machine-to-machine token.
- The OAuth M2M client uses that token to make an M2M call to the MCP server - the identity most at risk of privilege escalation, since impersonating it inherits whatever scope it was issued.
- Internal API layer: the MCP server sends an internal API request across the boundary into the Backend API, which queries the Database- the deepest layer, trusted implicitly by everything above it.

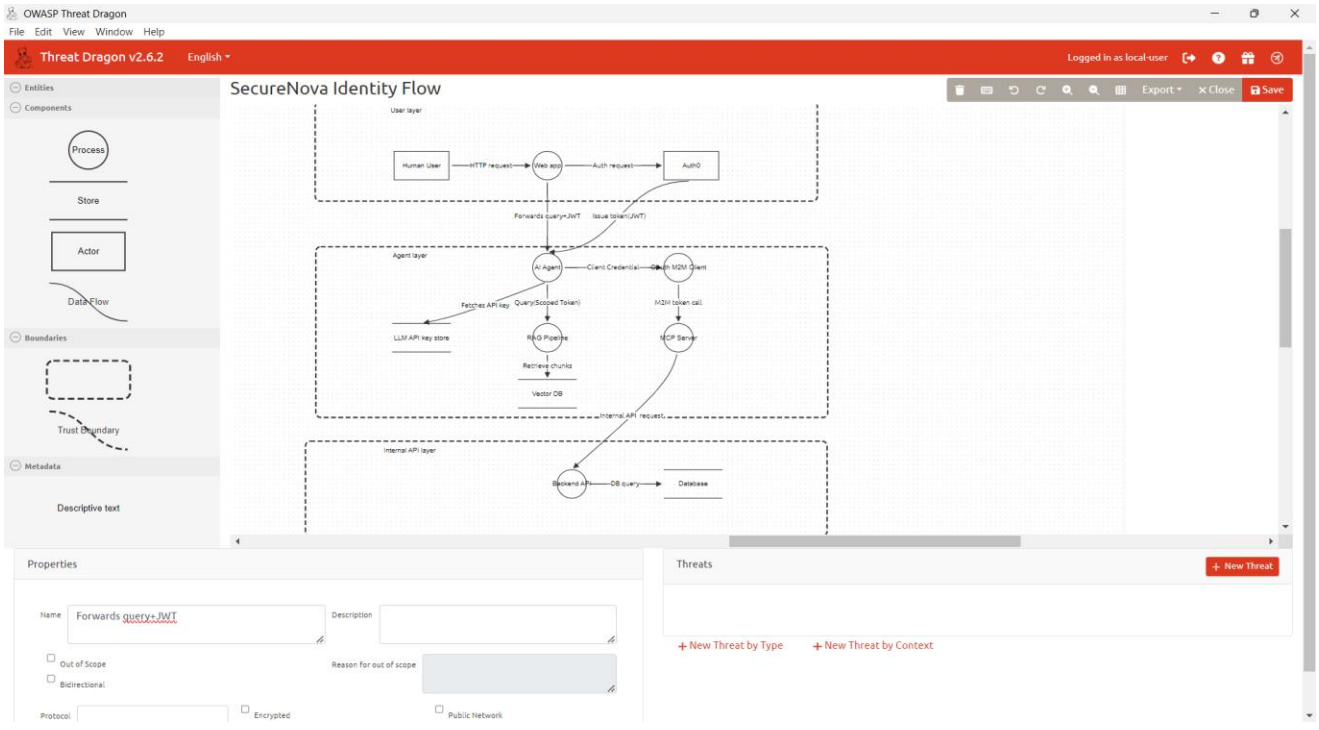
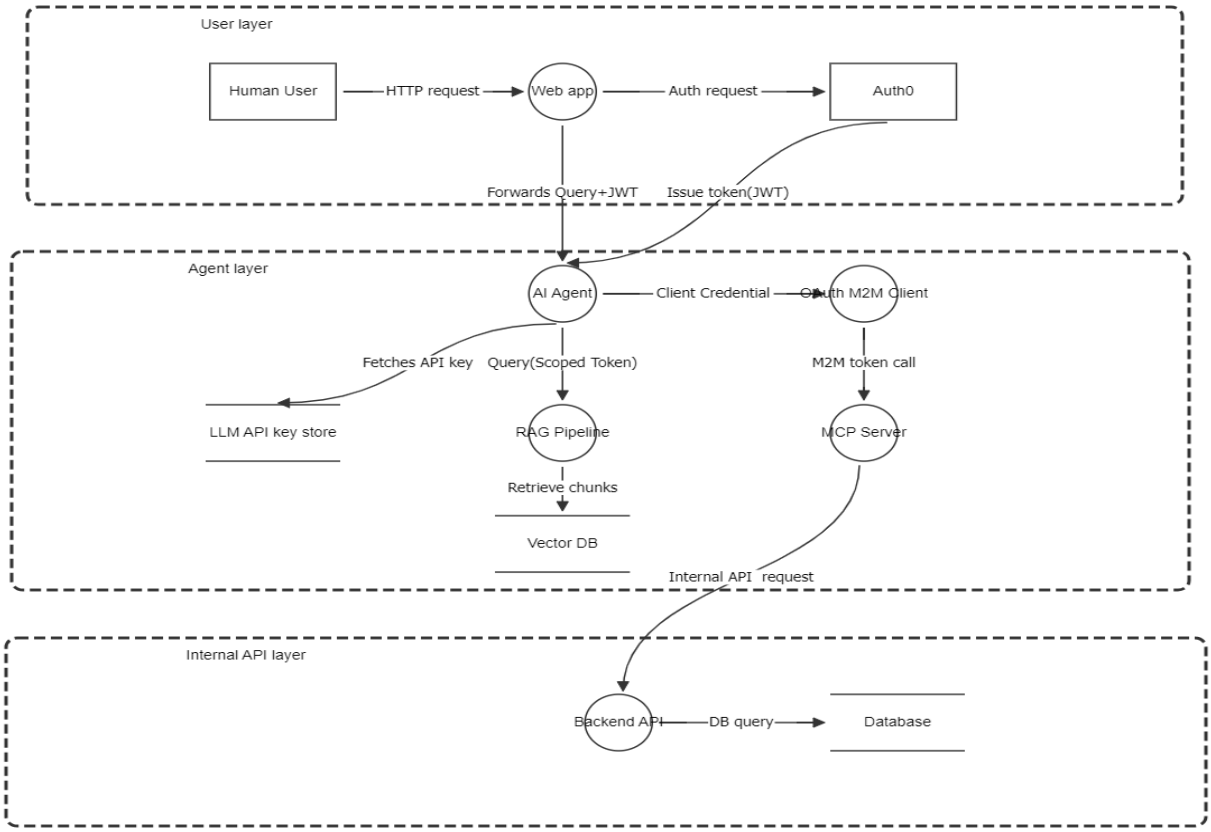


Figure 1: SecureNova AI identity platform — data-flow diagram with trust boundaries (OWASP Threat Dragon)



STRIDE Threats on the Web App → AI Agent Flow (Screenshot-2)

After building the data-flow diagram, I selected the Web app → AI Agent flow - the arrow that carries the customer's question along with their login token (JWT) into the AI system. I chose this flow because it is the first point where a request from the outside world is handed directly to the AI Agent, so anything wrong at this step affects everything the AI does afterward.

When I clicked on this arrow in Threat Dragon, only three STRIDE categories were available: Tampering, Information Disclosure, and Denial of Service.

I added the following three threats to this flow:

- Tampering - Prompt injection alters forwarded query. An attacker could inject hidden instructions into the customer's message before or during forwarding, causing the AI Agent to act on commands the real customer never gave.

- Information Disclosure - JWT exposed in transit or logs. The JWT riding along with this request could be exposed through weak transport encryption or plaintext logging, letting an attacker steal it and impersonate that user elsewhere.
- Denial of Service - Flooding the AI Agent with forwarded requests. An attacker could repeatedly trigger this flow, exhausting the AI Agent's compute or API quota and denying service to real customers.

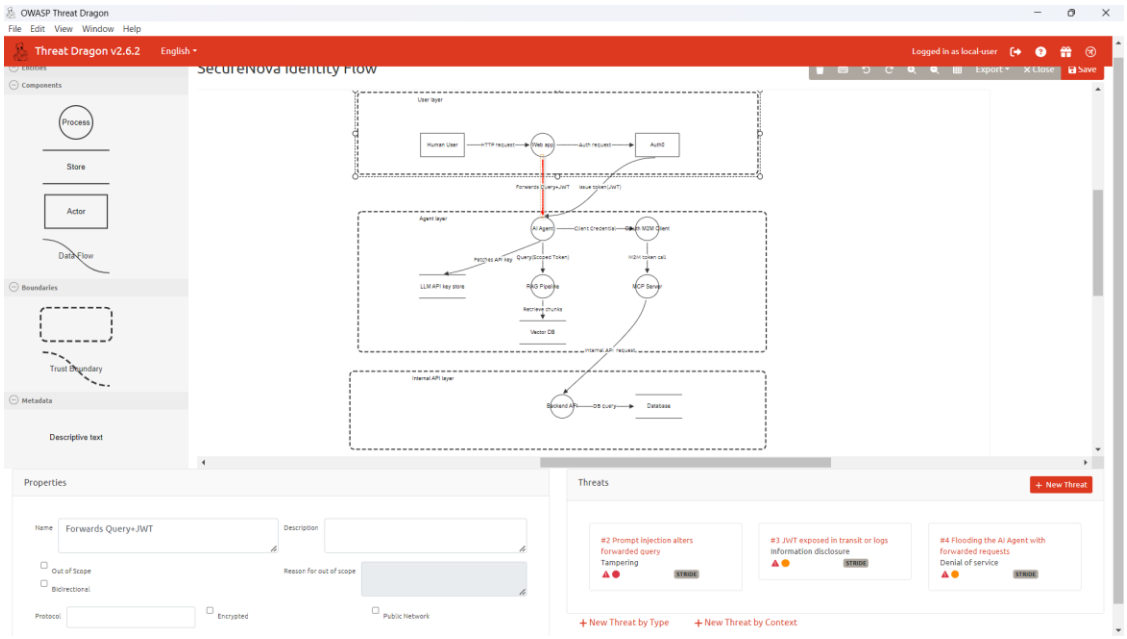
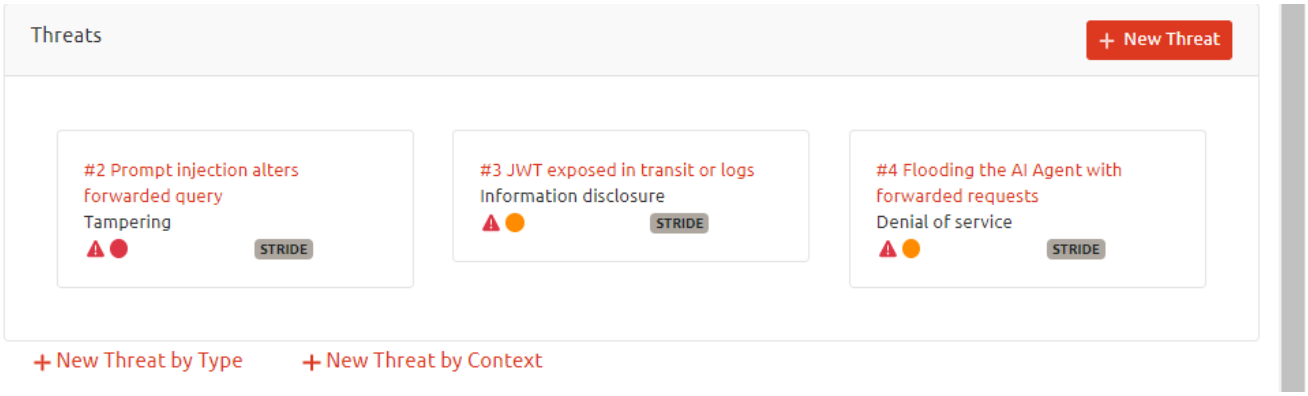


Figure 2: STRIDE threats populated on the Web app → AI Agent flow (OWASP Threat Dragon)



Attack Tree 1 - LLM API Key Exfiltration via Prompt Injection (Screenshot-3)

For this attack tree, I mapped every way an attacker could steal the LLM API key using nothing but prompt injection - no direct system access required. The goal sits at the top, and I broke it into three alternative (OR) paths: **Direct Prompt Injection**, where the attacker crafts a malicious prompt themselves, sends it straight to the agent, and overrides its instructions until it reveals the secret; **Indirect Prompt Injection**, where the attacker poisons a webpage, document, or email that the agent

later retrieves, so the agent follows hidden embedded instructions without the attacker ever contacting it directly; and **Injection via Tool/Agent Abuse**, where the attacker manipulates the agent into using one of its own tools in a way that exposes the secret or configuration it has access to.

All three paths converge on the same middle outcome - the API key becoming accessible. From there, an AND condition requires both that the key is actually exposed and that the attacker successfully receives or exfiltrates it, resulting in the final outcome at the bottom of the tree: the API key being stolen.

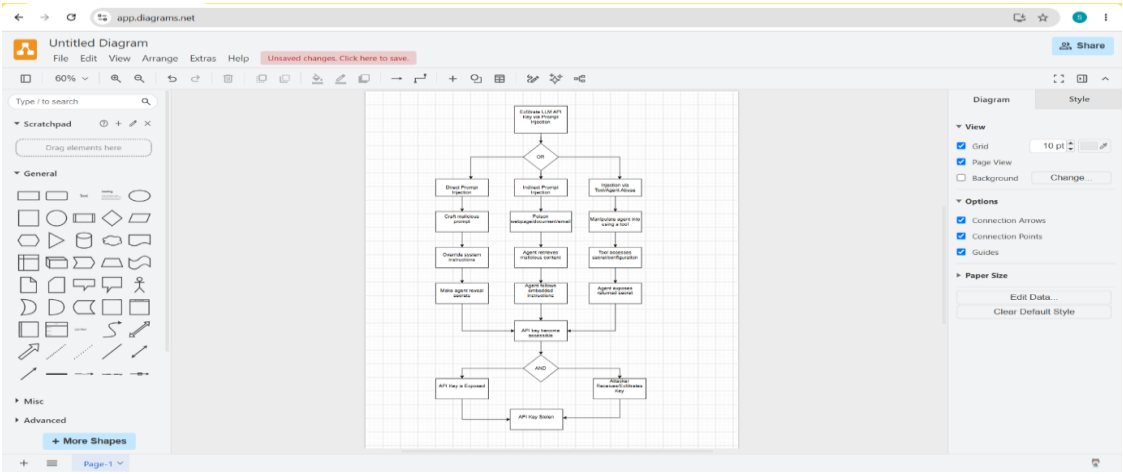
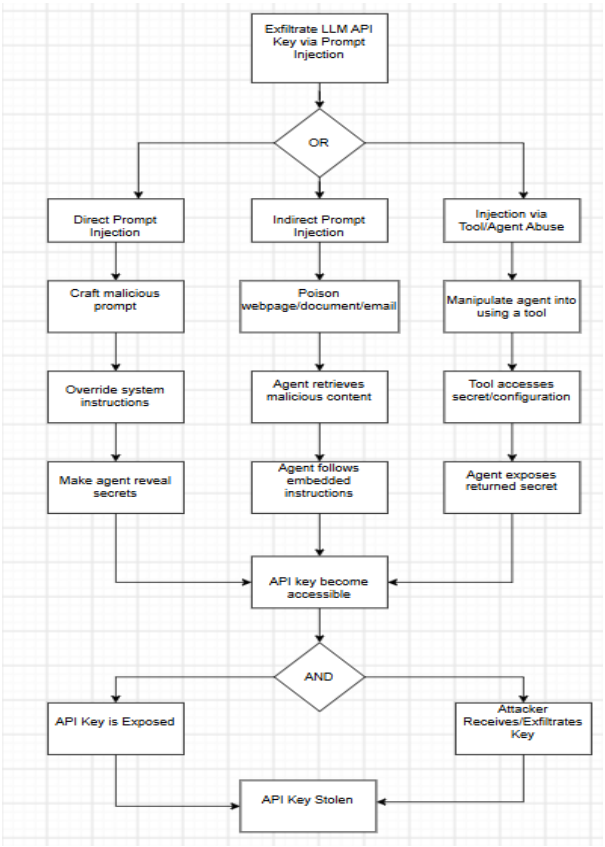
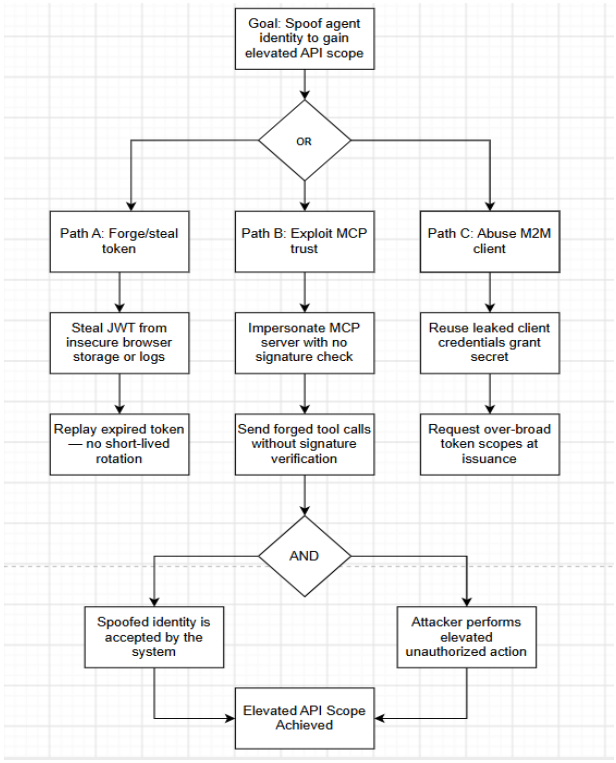


Figure 3: Attack Tree 1 — LLM API key exfiltration via prompt injection (draw.io)



Attack Tree 2 - Agent Identity Spoofing for Elevated API Scope (Screenshot-4)



This tree focuses on identity spoofing rather than prompt injection - the attacker's goal is to impersonate the AI agent, or the systems it trusts, to gain more API access than they should have. I broke the goal into three alternative paths: Path A, **forging or stealing a token**, where the attacker steals a JWT from insecure browser storage or logs and later replays an expired token because there is no short-lived token rotation enforced; Path B, **exploiting MCP trust**, where the attacker impersonates the MCP server itself - because it has no signature check and sends forged tool calls that are never verified; and Path C, **abusing the M2M client**, where the attacker reuses leaked OAuth client credentials to request over-broad token scopes at issuance.

Whichever path succeeds, an AND condition then requires that the spoofed identity is accepted by the system and that the attacker actually performs an elevated, unauthorized action together resulting in the final outcome: elevated API scope achieved.

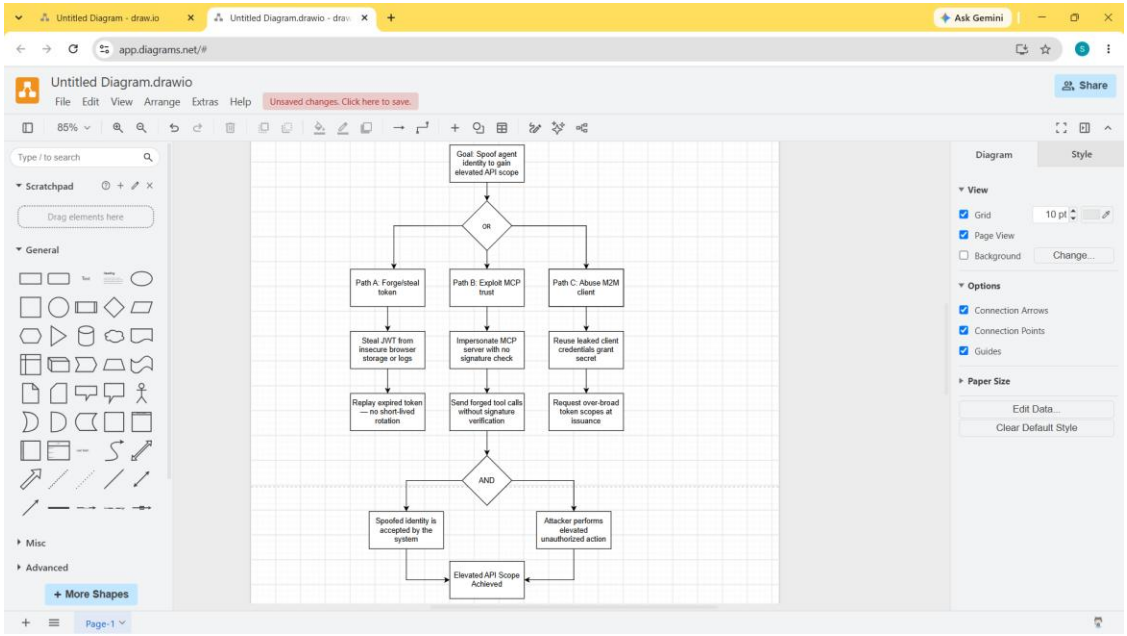


Figure 4: Attack Tree 2 — agent identity spoofing for elevated API scope (draw.io)

Attack Tree 3 — RAG Chunk Poisoning (Screenshot-5)

This tree models how an attacker could manipulate the AI agent's behavior indirectly, by poisoning the knowledge base the RAG pipeline retrieves from, rather than attacking the agent directly. I broke the goal into three alternative paths: Path A, injecting at the source, where the attacker submits a crafted document into an unvetted knowledge feed and hides malicious instructions in whitespace or metadata so they are not visually obvious; Path B, poisoning embeddings, where the attacker inserts adversarial vectors directly into the vector database, exploiting missing write-access controls; and Path C, exploiting ranking, where the attacker stuffs keywords to force their malicious chunk into the top-k retrieval results, exploiting the fact that there is no source trust scoring in retrieval.

All three paths converge on the same outcome a malicious chunk being present in the knowledge base.And condition then requires both that the agent actually retrieves the poisoned chunk&that it acts on the embedded instructions,resulting in the final outcome:the agent's behavior being manipulated.

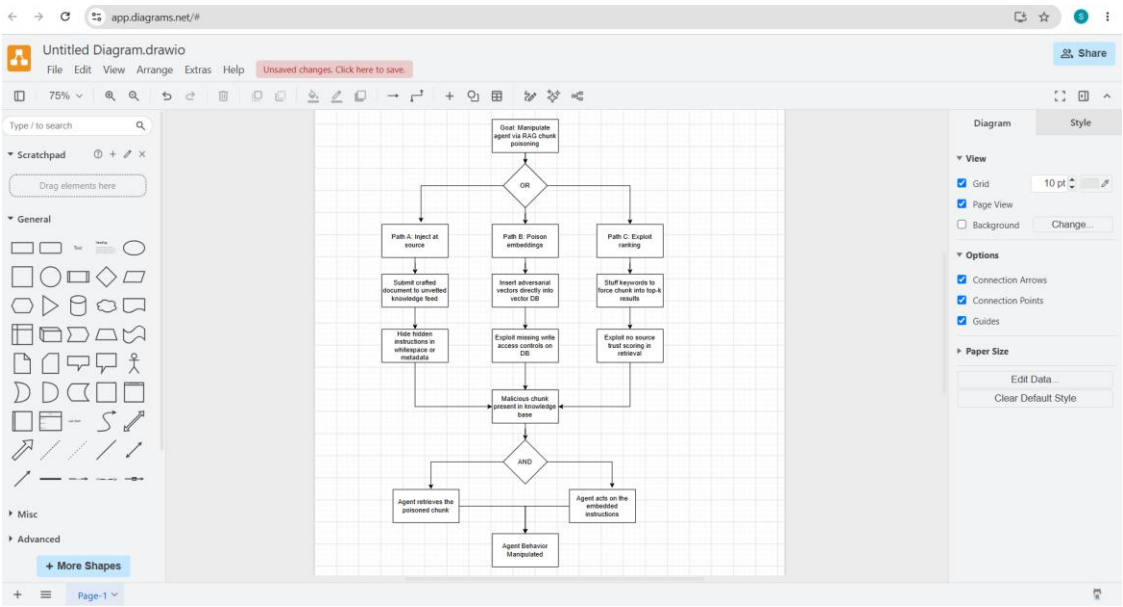


Figure 5: Attack Tree 3 — RAG chunk poisoning (draw.io)

STRIDE Threat Matrix (All Flows) (Screenshot-6)

After threatening individual flows one at a time in Threat Dragon, I consolidated every identity flow in the system into a single spreadsheet and checked each one against all six STRIDE categories Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Each row is one flow from the data-flow diagram (for example, Human User → Web App, or RAG Pipeline → Vector DB), and each column captures the specific way that flow could be attacked under that STRIDE category.

Building this matrix made it possible to compare threats across the entire system at once, rather than one flow at a time, and it directly feeds into the risk register in the next section every cell in this matrix that represents a real, credible threat became a row in that register.

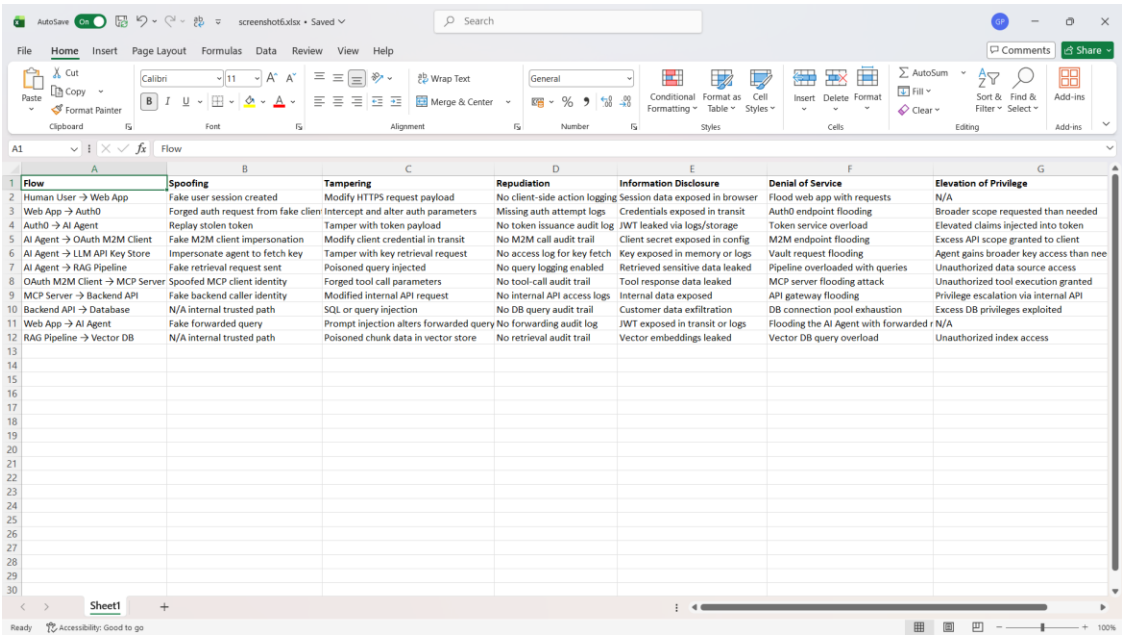


Figure 6: STRIDE threat matrix covering all identity flows in the platform (spreadsheet)

Flow	Spoofing	Tampering	Repudiation	Information Disclosure	Denial of Service	Elevation of Privilege
Human User → Web App	Fake user session created	Modify HTTPS request payload	No client-side action logging	Session data exposed in browser	Flood web app with requests	N/A
Web App → Auth0	Forged auth request from fake clien	Intercept and alter auth parameters	Missing auth attempt logs	Credentials exposed in transit	Auth0 endpoint flooding	Broader scope requested than needed
Auth0 → AI Agent	Replay stolen token	Tamper with token payload	No token issuance audit log	JWT leaked via logs/storage	Token service overload	Elevated claims injected into token
AI Agent → OAuth M2M Client	Fake M2M client impersonation	Modify client credential in transit	No M2M call audit trail	Client secret exposed in config	M2M endpoint flooding	Excess API scope granted to client
AI Agent → LLM API Key Store	Impersonate agent to fetch key	Tamper with key retrieval request	No access log for key fetch	Key exposed in memory or logs	Vault request flooding	Agent gains broader key access than nee
AI Agent → RAG Pipeline	Fake retrieval request sent	Poisoned query injected	No query logging enabled	Retrieved sensitive data leaked	Pipeline overloaded with queries	Unauthorized data source access
OAuth M2M Client → MCP Server	Spoofed MCP client identity	Forged tool call parameters	No tool-call audit trail	Tool response data leaked	MCP server flooding attack	Unauthorized tool execution granted
MCP Server → Backend API	Fake backend caller identity	Modified internal API request	No internal API access logs	Internal data exposed	API gateway flooding	Privilege escalation via internal API
Backend API → Database	N/A internal trusted path	SQL or query injection	No DB query audit trail	Customer data exfiltration	DB connection pool exhaustion	Excess DB privileges exploited
Web App → AI Agent	Fake forwarded query	Prompt injection alters forwarded query	No forwarding audit log	JWT exposed in transit or logs	Flooding the AI Agent with forwarded r	N/A
RAG Pipeline → Vector DB	N/A internal trusted path	Poisoned chunk data in vector store	No retrieval audit trail	Vector embeddings leaked	Vector DB query overload	Unauthorized index access

Risk Register (Screenshot-7)

This spreadsheet brings together every threat identified across the three attack trees and the STRIDE matrix into one prioritized list. Each threat is scored using Likelihood (1-5) × Impact (1-5) = Risk Score, then sorted from highest to lowest so the most dangerous issues are immediately visible at the top.

The highest-scoring threat, T-01 (craft a malicious prompt asking the agent to reveal system config or the API key), scored 25 - the maximum possible , because it requires no special access and directly causes credential exposure. The next several highest-ranked rows are also prompt-injection and RAG-poisoning related, confirming that prompt injection is the platform's single biggest risk category. Each threat also has an assigned owner (for example, AI Platform Security Lead, RAG Pipeline Owner, IAM Engineer) and a status, so this register can be tracked and acted on, not just read once.

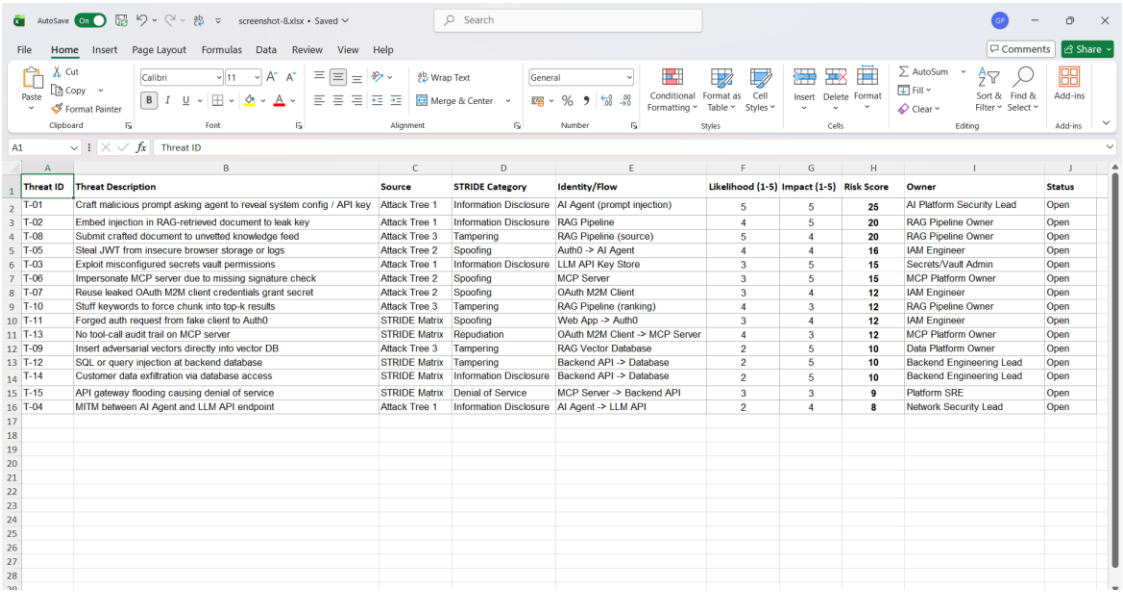


Figure 7: Risk register — every threat scored (Likelihood × Impact) and ranked, with an assigned owner (spreadsheet)

Threat ID	Threat Description	Source	STRIDE Category	Identity/Flow	Likelihood (1-5)	Impact (1-5)	Risk Score	Owner	Status
T-01	Craft malicious prompt asking agent to reveal system config / API key	Attack Tree 1	Information Disclosure	AI Agent (prompt injection)	5	5	25	AI Platform Security Lead	Open
T-02	Embed injection in RAG-retrieved document to leak key	Attack Tree 1	Information Disclosure	RAG Pipeline	4	5	20	RAG Pipeline Owner	Open
T-08	Submit crafted document to unvetted knowledge feed	Attack Tree 3	Tampering	RAG Pipeline (source)	5	4	20	RAG Pipeline Owner	Open
T-05	Steal JWT from insecure browser storage or logs	Attack Tree 2	Spoofing	Auth0 -> AI Agent	4	4	16	IAM Engineer	Open
T-03	Exploit misconfigured secrets vault permissions	Attack Tree 1	Information Disclosure	LLM API Key Store	3	5	15	Secrets/Vault Admin	Open
T-06	Impersonate MCP server due to missing signature check	Attack Tree 2	Spoofing	MCP Server	3	5	15	MCP Platform Owner	Open
T-07	Reuse leaked OAuth M2M client credentials grant secret	Attack Tree 2	Spoofing	OAuth M2M Client	3	4	12	IAM Engineer	Open
T-10	Stuff keywords to force chunk into top-k results	Attack Tree 3	Tampering	RAG Pipeline (ranking)	4	3	12	RAG Pipeline Owner	Open
T-11	Forged auth request from fake client to Auth0	STRIDE Matrix	Spoofing	Web App -> Auth0	3	4	12	IAM Engineer	Open
T-13	No tool-call audit trail on MCP server	STRIDE Matrix	Repudiation	OAuth M2M Client -> MCP Server	4	3	12	MCP Platform Owner	Open
T-09	Insert adversarial vectors directly into vector DB	Attack Tree 3	Tampering	RAG Vector Database	2	5	10	Data Platform Owner	Open
T-12	SQL or query injection at backend database	STRIDE Matrix	Tampering	Backend API -> Database	2	5	10	Backend Engineering Lead	Open
T-14	Customer data exfiltration via database access	STRIDE Matrix	Information Disclosure	Backend API -> Database	2	5	10	Backend Engineering Lead	Open
T-15	API gateway flooding causing denial of service	STRIDE Matrix	Denial of Service	MCP Server -> Backend API	3	3	9	Platform SRE	Open
T-04	MITM between AI Agent and LLM API endpoint	Attack Tree 1	Information Disclosure	AI Agent -> LLM API	2	4	8	Network Security Lead	Open

MITRE ATLAS Technique Mapping (Screenshot-8)

To ground Attack Tree 1 in a real, documented attack technique rather than a hypothetical one, I looked up the closest match on the MITRE ATLAS website (atlas.mitre.org) — a knowledge base of real-world adversarial techniques against AI systems, modelled after MITRE ATT&CK but specific to AI and machine learning.

The technique AML.T0051, LLM Prompt Injection, matches Attack Tree 1 directly: ATLAS describes it as an adversary crafting malicious prompts that cause the LLM to act in unintended ways, and it explicitly documents both Direct and Indirect injection as sub techniques - the exact same two categories I used as branches in my attack tree. This confirms that the attack path I modelled is not just theoretical; it is a technique actively tracked and documented in the industry-standard AI threat framework.

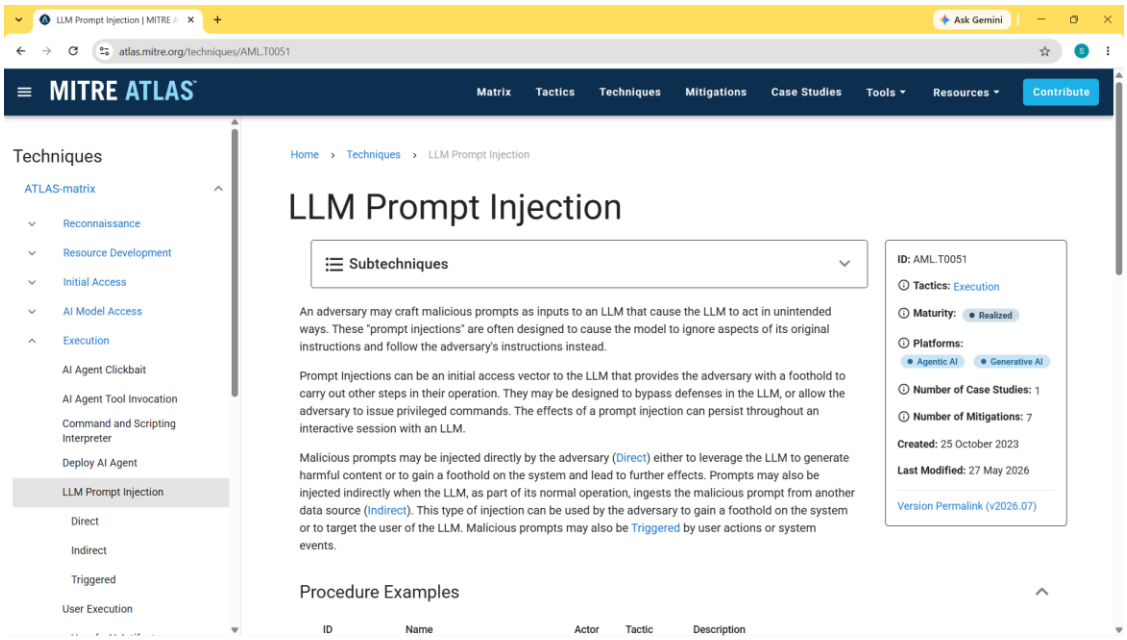


Figure 8: MITRE ATLAS technique page — AML.T0051, LLM Prompt Injection